

**Sri Sivasubramaniya Nadar College of Engineering, Chennai**  
(An autonomous Institution affiliated to Anna University)

|                     |                                                    |          |           |
|---------------------|----------------------------------------------------|----------|-----------|
| Degree & Branch     | M.Tech (Integrated) Computer Science & Engineering | Semester | V         |
| Subject Code & Name | ICS1512 – Machine Learning Algorithms Laboratory   |          |           |
| Academic Year       | 2025–2026 (Odd)                                    | Batch    | 2024–2029 |

**Experiment 9: Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning**

### Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA).
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activations.

Students must select and justify hyperparameters such as activation functions, cost functions, optimizers, learning rates, number of hidden layers, and batch sizes through systematic tuning.

### Dataset

**Download:** [English Handwritten Characters Dataset](#) Contains 3,410 images of handwritten characters (62 classes: 0–9, A–Z, a–z).

### Theory

#### PLA

- Step activation function. - Weight update rule:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

- Limitation: Only works well for linearly separable data.

#### MLP

- Architecture: Input  $\rightarrow$  Hidden Layer(s)  $\rightarrow$  Output. - Activation functions: ReLU, Sigmoid, Tanh. - Loss: Cross-Entropy (multi-class). - Optimizers: Gradient Descent, SGD, Adam. - Learns non-linear decision boundaries through backpropagation.

## Steps for Implementation

1. Preprocess dataset (resize, flatten, normalize).
2. Implement **PLA** from scratch:
  - Use step activation.
  - Apply weight update rule.
3. Implement **MLP**:
  - Choose hidden layers, neurons per layer, and activation functions.
  - Train using backpropagation.
  - Select cost function and optimizer through tuning.
4. Perform **hyperparameter tuning**:
  - Experiment with multiple settings for learning rate, batch size, optimizers, and activation functions.
  - Select the best-performing configuration and justify the choice.
5. Compare PLA and tuned MLP performance using evaluation metrics.

## Evaluation Metrics

- Accuracy, Precision, Recall, F1-score.
- Confusion Matrix.
- ROC Curves (micro/macro average).
- Training error vs epochs convergence curves.

## A/B Experiment Comparison

Students should present results for both PLA and MLP, clearly highlighting:

- Final chosen hyperparameters for MLP.
- Strengths and weaknesses of PLA vs MLP.
- Impact of hyperparameter tuning on convergence and accuracy.

## Observation Questions

- Why does PLA underperform compared to MLP?
- Which hyperparameters (activation, optimizer, learning rate, etc.) had the most impact on MLP performance?
- Did optimizer choice (SGD vs Adam) affect convergence?
- Did adding more hidden layers always improve results? Why or why not?
- Did MLP show overfitting? How could it be mitigated?

## Report Checklist

- Aim and Objective
- Preprocessing Steps
- PLA Implementation and Results
- MLP Implementation and Results
- Justification for Chosen Hyperparameters
- A/B Comparison (PLA vs MLP)
- Confusion Matrices and ROC Curves
- Observations and Analysis